

ADBC Scanner

Database drivers. Arrow batches. Ordinary SQL.



Install & load

```
INSTALL adbc_scanner FROM community;
LOAD adbc_scanner;
```

01 At a glance

Connect DuckDB to a database through an Arrow Database Connectivity driver. Use attached catalogs for joins and discovery, send backend SQL explicitly, or bulk-ingest a DuckDB query through the driver's Arrow interface.

ADBC driver → Arrow batches → DuckDB SQL

02 ATTACH API essentials

ATTACH ... (TYPE adbc)

Open a catalog; pass its alias to the functions below.

adbc_scan(db, sql)

Execute backend SQL; supply params and columns when needed.

adbc_scan_table(db, table)

Read a table with schema discovery and supported pushdown.

CALL adbc_execute(db, sql)

Run backend DDL or DML; return affected-row count.

adbc_insert(db, table, query)

Ingest Arrow batches; modes include create, append and replace.

adbc_schema(db, table)

Describe a named table, rather than an arbitrary query.

CALL adbc_clear_cache()

Refresh attached catalog metadata after remote DDL changes.

03 Keep in mind

Match the API revision

These examples target the v1.5 ATTACH API introduced in source revision 7e5210c. Older community builds use BIGINT connection handles instead. Install a matching build plus an ADBC driver manifest or shared library.

04 Query through a driver

MANUAL SETUP

```
ATTACH ':memory:' AS db (
  TYPE adbc, driver 'sqlite'
);
SELECT * FROM adbc_scan(
  'db', 'SELECT ? AS id',
  params := row(42),
  columns := {'id': 'BIGINT'}
);
```

Requires the ATTACH API build and SQLite ADBC driver. Returns id = 42.

05 Bulk load a DuckDB result

MANUAL SETUP

```
SELECT * FROM adbc_insert(
  'db', 'people',
  (SELECT 1::BIGINT AS id, 'Ada' AS name),
  mode := 'create'
);
SELECT * FROM adbc_scan_table('db', 'people');
SELECT * FROM adbc_schema('db', 'people');
DETACH db;
```

Continues the first example; ingestion modes depend on the driver.

06 In practice

Schemas and transactions

SQLite needs explicit columns for arbitrary scans. BEGIN / COMMIT controls the attachment's remote transaction; it does not create a distributed transaction. Use a file for SQLite catalog scans and transactions: separate :memory: connections are separate databases.

[Full docs & examples ↗](#) [Source ↗](#)